

7 EXPLORING PYTHON COLLECTIONS

7.1 List

In short, a list is a collection of arbitrary objects, somewhat like an array in many other programming languages but more flexible. Lists are defined in Python by enclosing a comma-separated sequence of objects in square brackets (`[]`), as shown below:

```
>>>nums = [3,30,6,7,23]
>>>nums
[3, 30, 6, 7, 23]
```

It is also possible to create an empty list by assigning a pair of empty square brackets (`[]`) or using `list()` constructor method, as follows:

```
>>>nums = []
>>>len(nums)
0
>>>nums = list()
>>>len(nums)
0
```

`len()` returns the number of items present in a list and returns 0, on passing an empty list. Out of these two approach to create an empty list, the second approach `list()` is slower because it requires looking up the name of the function, calling it, and then creating the list object in memory. In contrast, `[]` is like a "shortcut" that doesn't require so many intermediate steps to create the list in memory.

7.1.1 Why list?

Before we deep dive into the working principle of list let us first understand why list data structure is necessary and its purpose. In general, you should use lists when you need to and the purpose are listed below:

- **To keep your data in ordered form:** it is possible to store the data in a list in ordered form either as ascending or descending order or arbitrary.

- **Store a sequence of values:** Lists are a great choice when you need to store a sequence of related values. For example: list of names, mobile numbers, etc.
- **To update your records:** lists are mutable by nature means items of the list can be modified any number of times.
- **To access items randomly:** Lists allow quick and easy access to elements based on their index. List index started from 0, means the location of first item and supports negative indexing also.

There are sometimes we need to ignore list object to store some specific type of data:

- Don't use to store immutable data, as list is mutable in nature. Prefer tuple for immutable data and tuples are more memory efficient.
- If list is used to represent database records, then data can be easily modifiable. So, consider using a tuple or a data class.
- As list allows duplicate data, so it is not preferable to store unique and unordered values.
- Run many membership tests where item doesn't matter: In this case, consider using a set. Sets are optimized for this type of operation.
- To work with advanced array and matrix operations consider using NumPy's specialized data structures.
- For manipulate your data as a stack or queue consider using deque or Queue from collection module.

7.1.2 Accessing and Updating list items

You can access individual items from a list using the item's associated index. What's an item's index? Each item in a list has an index that specifies its position in the list. Indices are integer numbers that start at 0 and go up to the number of items in the list minus 1. The following syntax is used to access the list item using index:

```
>>>l1 = [4, 4, 2, 1, 4]
>>>l1[0]
4
```

Instead of accessing just one item, we can index multiple items by passing range of indexes. Some of the examples related to range of indexing is listed in Table 7.1. Consider a list `my_list = [1, 2, 3, 4, 5]`.

Table 7.1 Example of range of indexing with list slicing

Range of index	Explanation
<code>>>>mylist[1:4]</code>	Get elements from index 1 (inclusive) to 4 (exclusive)
<code>>>>my_list[:3]</code>	By leaving out the start value, the range will start at the first item. Get the first three elements
<code>>>>my_list[3:]</code>	Get the last two elements. By leaving out the end value, the range will go on to the end of the list
<code>>>>my_list[::2]</code>	Get elements at even indices
<code>>>>my_list[::-1]</code>	Reverse the list

<code>>>>my_list[::-2]</code>	Get elements at even indices in reverse order
<code>>>>my_list[1:4:2]</code>	Get elements from index 1 to 4 with a step of 2
<code>>>>my_list[2:4] = [6, 7]</code>	Replace elements at index 2 and 3 with 6 and 7
<code>>>>my_list[3:0:-1]</code>	Get elements from index 3 to 1 in reverse order
<code>>>>copy_list = my_list[:]</code>	Create a copy of the original list
<code>>>>my_list[-1:-4:-1]</code>	Get the last 3 elements in reverse order
<code>>>>my_list[1:-1]</code>	Get all elements except the first and last
<code>>>>my_list[:] + [6, 7]</code>	Create a new list by appending elements 6 and 7
<code>>>>my_list[2:7]</code>	Slicing beyond the end of the list is safe
<code>>>>my_list[-3:]</code>	Get the last three elements using a negative start index
<code>>>>my_list[1:4] = []</code>	Clear elements at index 1, 2, and 3
<code>>>>my_list[1:2] = [10]</code>	Change the second value by replacing it with two new values
<code>>>>my_list[1:3] = [3]</code>	Change the second and third value by replacing it with one value

7.1.3 Traversing list items

We can also access all the items of list using a loop till the length of the list. An example is shown below:

```
1 l1 = [4, 4, 2, 1, 4]
2 for i in range(0,len(l1)):
3     print(l1[i],end=' ')
```

`range(0, len(l1))` generates the indexes from 0 to (length -1) of the list. `l1[i]` accessing the items of `l1` using index the value of '`i`'. List items are also accessible using negative index. Negative indexing means start from the end, -1 refers to the last item, -2 refers to the second last item etc.

```
1 l1 = [4, 4, 2, 1, 4]
2 for i in range(-1,-len(l1)-1,-1):
3     print(l1[i],end=' ')
```

```
4 1 2 4 4
```

`range()` in line 2 generates the negative index starting from -1 to `-len(l1)` and `len[i]` accessing items from back of the list. Without index also items are accessible using a loop, as shown below:

```
1 my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
2 for x in my_list:
3     print(x, end=' ')
1 2 3 4 5 6 7 8 9
```

But the above style of accessing, doesn't allow us to update the list item. You can loop through the list

items by using a while loop as well. Use of `len()` function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes. Remember to increase the index by 1 after each iteration.

```
1 my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
2 i = 0
3 while i < len(my_list):
4     print(my_list[i], end=' ')
5     i = i + 1
```

In many situations, you'll need to know the index of the current item to be able to perform some computations. It's especially true when you're dealing with complex algorithms that operate on indices. In those cases, Python has you covered as well. It offers you the built-in `enumerate()` function, which you can use as in the following example:

```
fruits = ["banana", "Orange", "Kiwi", "cherry"]
for i, fruit in enumerate(fruits):
    print(f"{i} is the index of '{fruit}'")
```

Python provides many other tools that you can use when you're iterating through a list of values. For example, you can use `reversed()` to iterate over the list in reverse order:

```
1 fruits = ["banana", "Orange", "Kiwi", "cherry"]
2 for fruit in reversed(fruits):
3     print(fruit, end=' ')
```

cherry Kiwi Orange banana

The `reversed()` function to traverse your list of fruits in reverse order, which might be a common requirement in your code. Another common need is to traverse the list in sorted order. For this, you can use `sorted()` as in the code below:

```
1 numbers = [2, 9, 5, 1, 6]
2 for number in sorted(numbers):
3     print(number, end=' ')
```

1 2 5 6 9

The `zip()` function, which allows you to iterate over multiple lists in parallel. The `zip()` function returns an iterator of tuples. The elements of each tuple come from the input iterables. In this example, the tuples combine items from three lists integers, letters, and floats.

```

1 integers = [1, 2, 3]
2 letters = ["a", "b", "c"]
3 floats = [4.0, 5.0, 6.0]
4
5 for i, l, f in zip(integers, letters, floats):
6     print(i, l, f)

```

```

1 a 4.0
2 b 5.0
3 c 6.0

```

7.1.4 Concatenating and Repeating

In this Section you'll learn how these two operations are performed with lists and how you can use them in your code. It is an useful feature of Python's list is that it supports the following two operations:

- Concatenation, which uses the plus operator (+)
- Repetition, which uses the multiplication operator (*)

Concatenation consists of joining two things together. In this case, you'd like to concatenate two lists, which you can do using the plus operator (+).

```

>>>[0, 1, 2, 3] + [4, 5, 6] + [7, 8, 9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```

Whenever you use the concatenation operator, you get a new list object as a result. Consider the following example and keep an eye on the identity of **digits** list, that after every concatenation its identity get changed:

```

>>>digits = [0, 1, 2, 3, 4, 5]
>>>id(digits)
3028851826880
>>>digits = digits + [6, 7, 8, 9]
>>>id(digits)
3028851824384

```

You can only concatenate a list with another list. If you try to concatenate a list with something else, then Python will raise a `TypeError` exception:

```
>>>[0, 1, 2, 3, 4, 5] + (6, 7, 8, 9)
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    [0, 1, 2, 3, 4, 5] + (6, 7, 8, 9)
TypeError: can only concatenate list (not "tuple") to list
```

The concatenation operator has an augmented variation, which uses the `+=` operator. The augmented concatenation operator works on an existing list. It takes a second list and adds its items, one by one, to the end of the initial list. The operation is a shortcut to something like `digits = digits + [6, 7, 8, 9]`. However, it works a bit differently. Unlike the regular concatenation operator, the augmented variation mutates the target list in place rather than creating a new list:

```
>>>digits = [0, 1, 2, 3, 4, 5]
>>>id(digits)
3028851821952
>>>digits += [6, 7, 8, 9]
>>>id(digits)
3028851821952
```

In this example, the `id()` function returns the same value in both calls, meaning that you have a single list object instead of two.

Repeating the Content of a List

Repetition consists of cloning the content of a given list a specific number of times. You can achieve this with the repetition operator (`*`), as follows:

```
>>>["A", "B", "C"] * 3
['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C']
```

In the above example, you repeat the content of a list three times and get a new list as a result. The right-hand operand is an integer representing the number of times that you want to repeat the list's content.

7.1.5 Comparing Two List

You can also face the need of comparing two lists and it can be done using the standard comparison operators. All these operators work by making item-by-item comparisons within the two involved lists, as follows:

```
>>>[2, 3] == [2, 3]
True
>>>[5, 6] != [5, 6]
```

```

False
>>>[5, 6, 7] < [7, 5, 6]
True
>>>[5, 6, 7] > [7, 5, 6]
False
>>>[4, 3, 2] <= [4, 3, 2]
True
>>>[4, 3, 2] >= [4, 3, 2]
True

```

Lists with different lengths can also be compared and in this case all the items of the first list (left operand) will be compared with the right-hand side operand, based on the given operator.

```

>>>[5, 6, 7] < [8]
True
>>>[5, 6, 7] == [5]
False

```

In the first expression, you get `True` as a result because 5 is less than 8. That fact is sufficient for Python to solve the evaluation. In the second example, you get `False`. This result makes sense because the lists don't have the same length, so they can't be equal.

7.1.6 Built-in Methods

Python has a set of built-in methods that you can use on lists. To add an item to a list the `append()` and `insert()` method can be called on a list object.

```

>>>nums = [1,2,3]
>>>nums.append(6)
>>>nums
[1, 2, 3, 6]
>>>nums.insert(1,7)
>>>nums
[1, 7, 2, 3, 6]

```

`nums.append(6)`, appends an object at last and `nums.insert(1,7)` requires two parameters. The first parameter is '1', that specifies the index or location where to be inserted the given object '7' as second parameter. To remove an item or all from a list object, we can use `clear()`, `pop()`, and `remove()` methods, as follows:

```
>>>nums.remove(2)
>>>nums
[1, 7, 3, 6]
```

To remove any specific item from a list, we can pass any value to `nums.remove(2)`, if the item is present, else Python will raise a `ValueError`. To handle this issue one can first check the membership of an item, if present then it can be removed.

```
#10 not present, no error is raised
>>>if 10 in nums: nums.remove(10)
>>>nums
[1, 7, 3, 6]

#7 remove
>>>if 7 in nums: nums.remove(7)
>>>nums
[1, 3, 6]
```

To remove any item from a specific location `pop()` method is used, by passing the index to it. If the given index is equal or more than the length of a list, it will raise an `IndexError`. To handle this, you can first check the length of the list and then pass an index value lesser than the length.

```
>>>index = 9
>>>nums.pop(index) if index < len(nums) else print('Invalid index')
Invalid index
>>>index = 2
>>>nums.pop(index) if index < len(nums) else print('Invalid index')
6
>>>nums
[1, 3]
```

Using `.clear()` all the items of a list can be removed and the list object will be empty.

```
>>>nums.clear()
>>>nums
[]
```

List `index()` method searches for a given element from the start of the list and returns the index of the first occurrence. Like `remove()`, `index()` can also raise a `ValueError`, if the given value is not present in the list object.


```
>>>nums = [2,4,5,6,7]
>>>nums.index(3)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    nums.index(3)
ValueError: 3 is not in list
>>>nums.index(4)
1
```

ValueError can be taken care by using an extra check that we did earlier for `remove()`. Instead of searching all over the complete list, we can also pass a range of indexes using start and end value to the `index()` method, as follows:

```
>>>nums = [1, 2, 3, 4, 1, 1, 1, 4, 5]
>>>nums.index(4,3,6)
3
```

The code uses Python's built-in `.index()` method to find the index of the number 4 within a specific range of a list. the index method is searching for the first occurrence of the number 4 between indices 3 and 5 (because 6 is exclusive). Since 4 is found at index 3, which is within the specified range, the method returns 3 as the output. If the value 4 wasn't found within the specified range (3 to 5), a ValueError exception would be raised.

The `.count()` method in Python is a built-in function used to count the number of times a specific element appears in a list.

```
>>>points = [1, 4, 2, 9, 7, 8, 9, 3, 1]
>>>points.count(9)
2
```

The above code uses the `.count()` method to find the frequency of 9 within points. The count method then searches the entire points list and returns the number of times it finds the value 9.

The `.copy()` method returns a shallow copy of the original list. It creates a new list object that contains the same elements as the original list, but any modifications made to the new list will not affect the original list.

```
>>>l1 = [4, 4, 2, 9, 7, 8]
>>>l2 = l1.copy() #creates a shallow copy
>>>l2[0] = 1
>>>l2
[1, 4, 2, 9, 7, 8]
>>>l1
[4, 4, 2, 9, 7, 8]
```

After the copy operation, first item at index 0, in list `l2` is updated, but original list `l1` didn't get effected. Instead of `.copy()`, if the assignment operator is used, then it creates a deep copy of the original list. Means changes made inside the new list, will modifying the actual list. See the illustration below:

```
>>>l1 = [4, 4, 2, 9, 7, 8]
>>>l2 = l1 #creates a deep copy
>>>l2[0] = 1
>>>l1
[1, 4, 2, 9, 7, 8]
```

```
>>>l2
[1, 4, 2, 9, 7, 8]
```

The `.extend()` method adds the specified list elements (or any iterable) at the end of the current list. In the illustration, items of `l2` added at the end of `l1`, on which `.extend()` is called.

```
>>>l1 = [4, 4, 2]
>>>l2 = [1, 4]
>>>l1.extend(l2)
>>>l1
[4, 4, 2, 1, 4]
```

The `reverse()` method is used to reverse the order of elements in a list. It is an in-place reversal method, meaning it modifies the original list and does not create a new one, making it memory-efficient.

```
>>>l1.reverse()
>>>l1
[4, 1, 2, 4, 4]
```

The `sort` method is used to sort elements in a list. By default, it sorts the list in ascending order. You can optionally customize the sorting behavior using two arguments:

- `reverse`: This is a boolean value (True or False). If set to True, the list will be sorted in descending order.
- `key`: This is a function that takes one element from the list as input and returns a value to be used for comparison during sorting. This allows you to define custom sorting criteria.

```
>>>l1 = [4, 4, 2, 1, 4]
>>>l1.sort() #ascending order
>>>l1
[1, 2, 4, 4, 4]
>>>l1.sort(reverse=True) #descending order
>>>l1
[4, 4, 4, 2, 1]
```

The `key` parameter allows you to specify a function that computes a value for each element in the list, and the list is then sorted based on the computed values rather than the elements themselves. See the illustration:

```
>>>cars = ['Ford', 'BMW', 'Volvo']
>>>cars.sort(key=len)
>>>cars
['BMW', 'Ford', 'Volvo']
```

In the above example list is sorted based on the length of each string. To do that the `len()` function is used as the `key` parameter, which computes the length of each string. The `sort()` function then uses these computed lengths to sort the list. if you want a case-insensitive sort function, use `str.lower()` as a `key` function

```
>>>fruits = ["banana", "Orange", "Kiwi", "cherry"]
```

```
>>>fruits.sort(key = str.lower)
>>>print(fruits)
['banana', 'cherry', 'Kiwi', 'Orange']
```

Program 7.1: Design a user-defined function to sort a list of fruit names based on the last letter of each fruit name.

```
# Define a function to sort by the last character of the string
def sort_by_last_letter(word):
    return word[-1]

words = ["apple", "banana", "cherry", "date", "apricot"]

# Sort the list by the last character of each word
words.sort(key=sort_by_last_letter)
print(words)

['banana', 'apple', 'date', 'apricot', 'cherry']
```

sort_by_last_letter() function takes a single argument word (which is a string in this case) and returns the last character of the word. This function essentially defines the sorting criteria. **words.sort(key=sort_by_last_letter)** line sorts the words list using the sort method. The key argument specifies the **sort_by_last_letter** function. This means the sorting will happen based on the last letter of each word (as determined by the **sort_by_last_letter** function). As you can see, the words are sorted alphabetically based on their last characters (t, a, y, e, and e).

7.1.7 List with Functional Tools

You can also take advantage of some Python functional programming tools, such as **map()** and **filter()**, to traverse a list of values. These functions have an internal loop that iterates over the items of an input iterable and returns a given result.

The **map()** function takes a transformation function and an iterable as arguments. Then it returns an iterator that yields items that result from applying the function to every item in the iterable.

```
>>>numbers = ["2", "9", "5", "1", "6"]
>>>numbers = list(map(int, numbers))
>>>numbers
[2, 9, 5, 1, 6]
```

In the above illustration, **map()** applies **int()** to every item in numbers in a loop. Because **map()** returns an iterator, to the **list()** constructor to consume the iterator and show the result as a list.

If you need to filter out some values from an existing list, then you can use the built-in **filter()** function. This function takes two arguments: a predicate function and an iterable of data. Then it returns an iterator that yields items that meet a given condition, which the predicate function tests for. Here's how **filter()** works in practice:

```
>>>integers = [20, 31, 52, 6, 17, 8, 42, 55]
>>>even_numbers = list(filter(lambda number: number % 2 == 0, integers))
>>>even_numbers
[20, 52, 6, 8, 42]
```

In the above illustration `map()` is used when you want to transform every item in an iterable using a function and collect the results. `filter()` is used when you want to select and retain specific elements from an iterable based on a condition. The built-in `min()` and `max()` takes a list as argument and returns the smallest and largest of any list.

```
>>>l1 = [3,4,5,12,7]
>>>min(l1)
3
>>>max(l1)
12
```

Sometimes in programming it is necessary to check the truthiness of a list and for that Python offers two built-in functions as `any()` and `all()`. Refer the programming example 7.2 to understand the working of these functions with list object.

Program 7.2: Suppose you want to schedule interviews for all the candidates, who meet any or all of the following criteria, (a) Knows Python; (b) Have experience five or more years; and (c) Have a degree

```
applicants = [
    {
        "name": "Devon Smith", "programming_languages": ["c++", "ada"],
        "years_of_experience": 1, "has_degree": False,
        "email_address": "devon@email.com"},
    {
        "name": "Susan Jones", "programming_languages": ["python", "javascript"],
        "years_of_experience": 5, "has_degree": True,
        "email_address": "susan@email.com"},
    {
        "name": "Sam Hughes", "programming_languages": ["java"],
        "years_of_experience": 4, "has_degree": True,
        "email_address": "sam@email.com"},
]
for applicant in applicants:
    meet_criteria = []
    meet_criteria.append("python" in applicant["programming_languages"])
    meet_criteria.append(applicant["years_of_experience"] >= 5)
    meet_criteria.append(applicant["has_degree"])
    if any(meet_criteria):
        print(f'{applicant["name"]}')
```

Susan Jones
Sam Hughes

At first a list of dictionaries namely `applicants` are created, where each candidate information is

preserved as dictionary with information like, **name**, **programming_languages**, **year_of_experience** and **email_address**. Then record of each candidate are checked for shortlisting them based on the required criteria, if any condition matched then program will print the candidate names.

For loop is used to iterate each candidate records and through membership operator **programming_languages**, information is checked for a candidate having knowledge for python or not. Further **years_of_experience** is checked for the candidate having experience more than or equal to 5. At last candidate degree has been recorded and all these information is kept in a list **meet_criteria**.

The **any()** functions returns true because 'Susan Jones' has 'python' as known language and 'Sam Hughes' have a degree. Now if we simply want to shortlist all the candidates matched with all the criteria, then instead of **any()**, **all()** will be used, in the if expression, as shown below:

```
if all(meet_criteria):
    print(f'{applicant["name"]}')
```

Susan Jones

From the output we can check only 'Susan Jones' has matched all the criteria and **any()** returns True, if all the items of a given list are True.

7.1.8 The del statement

del statement is used to remove an item from a list based on the given index, instead of value. This differs from the **pop()** method which returns a value, that we have seen earlier. The **del** statement can also be used to remove slices (a sub-part of the list) from a list or clear the entire list. **del** statement also useful to erasing a list object from memory, and on referencing such object after del operation, will raise a **NameError**, see the illustration given below:

```
>>>a = [-1, 1, 66.25, 333, 333, 1234.5]
>>>del a[0] #delete an item at index 0
>>>a
[1, 66.25, 333, 333, 1234.5]
>>>del a[2:4] #delete items from index 2 to 4 (not including)
>>>a
[1, 66.25, 1234.5]
>>>del a[:] #to delete all the list items
>>>a
[]
>>> del a #delete the complete list from memory
>>> a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined
```

7.1.9 List Comprehension

List comprehension is a syntactic construct used to create a new list from an existing list. It follows the form of the mathematical set-builder notation (set comprehension) as distinct from the use of map and filter functions. Consider the following example in set-builder notation:

$$S = \{2 \cdot x | x \in \mathbb{N}, x^2 > 3\}$$

or often

$$S = \{2 \cdot x : x \in \mathbb{N}, x^2 > 3\}$$

This can be read as, **S** is the set of all numbers, 2 times of **x**, where **x** is an element or member of set natural number $\mathbb{N}$, and **x** square is greater than 3. The smallest natural number is 1 and the condition $1^2 > 3$, fail to satisfy the condition so it is not included in the new list, else all other numbers are added. The given set-builder notation can be annotated as follows,

$$S = \left\{ \underbrace{2 \cdot x}_{\text{output expression}} \mid \underbrace{x}_{\text{variable}} \in \underbrace{\mathbb{N}}_{\text{input set}}, \underbrace{x^2 > 3}_{\text{predict}} \right\}$$

where,

- **x** is the variable representing numbers of an input set or a list.
- $\mathbb{N}$ represents the input set, in this example we have taken the set of natural number.
- $x^2 > 3$, is a predictor used as a filter out some items from existing list or set
- $2 \cdot x$ is an output expression producing member for new list or set
- Symbol **{}** indicates that the result is a set
- The vertical bar **|** is read as such that
- Comma (,) separates the predict and read as AND

A list comprehension has the same syntactic components to represent generation of a list in order from an input list or iterator:

```
newlist = [expression for item in iterable if condition == True]
```

In the above syntax of list comprehension,

- The **condition** is similar to the predict rule as shown above and it is optional and used when some selected items are picked
- An **iterable** can be anything from string, list, tuple, dictionary, tuple and set.
- **expression** can have any valid python expression

Let's explore list comprehension with some examples and generate a new list from a given list **fruits**, containing only the fruits with the letter "a" in the name.

```
1 fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
2 newlist = [name for name in fruits if "a" in name]
3 print(newlist)
```

```
['apple', 'banana', 'mango']
```

In line 2 of the above illustration, if letter "a" is in **name** having the value as a fruit names from **fruits** iterator, is the predictor used and worked as filter out some fruits name from the given iterable object **fruits** and fruit names are used as part of the **expression**. If we want to generate the same list without

list comprehension it needs more than one line of source code and a conditional statement need to be put inside for statement, as follows:

```
1 fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
2 newlist = []
3
4 for fruit_name in fruits:
5     if "a" in fruit_name:
6         newlist.append(x)
7
8 print(newlist)
```

It is not necessary to use any predicate rules of expression as part of list comprehension. The next example, generates a new list without the using of any predicate rule. But as part of the expression each item of the given list is squared with the expression `number**2`.

```
>>>numbers = [3,4,5,6,8]
>>>nums = [number**2 for number in numbers]
>>>print(nums)
[9, 16, 25, 36, 64]
```

In list comprehension a condition is act as a filter that only accepts the items, if it evaluates to True. See the next illustration:

```
>>>numbers = [3,4,5,6,8]
>>>nums = [number**2 for number in numbers if number % 2 == 0]
>>>print(nums)
[16, 36, 64]
```

The condition `if number % 2 == 0`, will be True for all the numbers are even, making a new list containing the square of all the even numbers. As part of the iterable object anything can be used for a sequence type. It is also possible to generate a sequence through `range()` function:

```
>>>[x+1 for x in range(10)]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

You can also use a string as an iterable and generating a list of letters of a given string, using comprehension:

```
>>>[char for char in "Python"]
['P', 'y', 't', 'h', 'o', 'n']
```

The expression `char` is the current item in the iteration, but it is also the outcome, which you can manipulate before it ends up like a list item in the new list. The next example converting all the fruits name to upper case:

```
>>>fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
>>>[x.upper() for x in fruits]
['APPLE', 'BANANA', 'CHERRY', 'KIWI', 'MANGO']
```

You can also generate a new list containing all the same item of equal length of existing one, as shown below:

```
>>>fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
>>>['banana' for x in fruits]
['banana', 'banana', 'banana', 'banana', 'banana']
```

The expression can also contain a condition, not like a filter, but as a way to manipulate the outcome. The given condition, `if x != "banana"` checks if the current element (`x`) is not equal to "banana". If it's True, then the current element (`x`) itself is included in the new list. If it's False (`x == "banana"`), then "orange" is included in the new list instead.

```
>>>fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
>>>[x if x != "banana" else "orange" for x in fruits]
['apple', 'orange', 'cherry', 'kiwi', 'mango']
```

7.1.10 2-d List

Working with 2-d list includes the understanding of the structure, accessing elements, and efficiently manipulating data in a two-dimensional grid. When working with structured data or grids, 2-d lists can be useful. A 2-d list is a collection of a list of lists, which represents a table-like structure with rows and columns. In Python matrices are represented as 2-d list and python offers many ways to create a 2-d list. Let's see our first example of creating a 2-d list with 2 rows and 2 columns:

```
>>>A = [[1,2],[3,4]]
>>>A
[[1, 2], [3, 4]]
```

The 2D list is enclosed within double square brackets `[]`. Inside the outer brackets, there are two inner square brackets `[]`, each containing two elements. The memory view of a 2-d list is like a grid only and to access any item from a 2-d list we need to pass two indexes, one for row and other for column. See the next tabular view to understand the indexing of a 2-d list.

	0	1
0	1 [0][0]	2 [0][1]
1	3 [1][0]	4 [1][1]

In the above grid numbers on top row and left-most column represents column and row indexes. Numbers inside the square `[]` brackets, such as `[0][0]` represents the location of the first item. To access of these items, we need to combine row and column indices with the list name, as follows:

```
>>>A[0][0]
1
>>>A[1][0]
3
```


If a 2-d list is small in size then it is easy to be declared with values. But if we want populate a 2-d list based on user input, we need to do it row wise and keep appending it to the main 2-d list. Here is an example of taking input from user of 3×3 matrix.

```

1 MAT = []
2 rows, cols = (3,3)
3 #Taking input rowwise
4 for i in range(0,rows):
5     row = input(f'Enter data:').split()
6     row = [int(item)for item in row]
7     MAT.append(row)
8
9 #print 2-d list in matrix form
10 for i in range(0,rows):
11     for j in range(0, cols):
12         print(MAT[i][j], end=' ')
13     print()

```

```

Enter data:1 2 3
Enter data:4 5 6
Enter data:7 8 9
1 2 3
4 5 6
7 8 9

```

The various components of the above illustration are described below:

- Initially an empty 1-d list **MAT = []** is declared at line 1, which will eventually hold the 2D data.
- In line 2, **rows, cols = (3, 3)** assigns the values 3 to both rows and cols variables. These represent the dimensions of the 2D list, creating a 3x3 matrix in this case.
- The for loop iterates three times (0, 1, 2) based on the value of rows. prompts the user to enter data for each row. The **.split()** method splits the input string into a list of elements (assuming space-separated values).
- The list comprehension at line 6, converts each item in the row list from string to integer. **MAT.append(row)**, at line 7, adds the processed row (now a list of integers) to the MAT list. This builds the 2D structure row by row.
- The outer loop at line 10, iterates three times again to control the rows when printing. The inner loop at line 11, iterates three times within each outer loop iteration to control the columns when printing.

Like 1-d list a 2-d list can also be created through nested list comprehension. Personally, I didn't prefer working with 2-d list as list comprehension as syntax are complex. But as per anyone interest you can use it while declaring a 2-d list and latter values can be appended by accessing individual locations. The next illustration demonstrates the creation of 2-d list through list comprehension.

```
# Create a 2D list with 3 rows and 4 columns filled with zeros
```

```

rows = 3
cols = 4
matrix = [[0 for _ in range(cols)] for _ in range(rows)]

# Display the 2D list
for row in matrix:
    print(row)

```

The above illustration demonstrates the use of list comprehension for creating a 2D list filled with zeros. Here's a breakdown:

- In the list comprehension, outer **for _ in range(rows)** loop iterates three times (0, 1, 2) to create three rows.
- Within the outer loop, the inner **[0 for _ in range(cols)]** list comprehension creates a sub-list (representing a row) filled with zeros.
- The inner loop iterates four times (0, 1, 2, 3) to create four zeros for each row.
- The entire expression creates a nested structure, building the 2D list with rows containing zeros.

7.2 Tuple

Tuple is one of the built-in data types in Python used to store collections of data like list, set and dictionary. A is an immutable ordered sequence of items, means existing objects in the sequence cannot be modified, and new elements cannot be added once the sequence is created. Two ways we can create a tuple, one using **tuple()** constructor method and other using a pair of empty parentheses “()”, as shown below:

```

>>>t = ()
>>>type(t)
<class 'tuple'>
>>>t = tuple()
>>>type(t)
<class 'tuple'>

```

A tuple consists of several values separated by commas. Tuple items are ordered, unchangeable, and allow duplicate values. Tuple items are indexed like list and the first item is accessed using index **[0]**.

```

>>>t = ("apple", "banana", "cherry")
>>>t[0]
'apple'

```

A tuple object can iterate through a loop using for or while. We can also use **range()** function to generate the index of every item, as shown below:

```

fruits = ("apple", "banana", "cherry")
#Accessing each item
for fruit in fruits:
    print(fruit,end=' ')

```

```
#Accessing items using index
for i in range(0,len(fruits)):
    print(fruits[i], end=' ')
```

Membership of any item can also be tested using membership operators, as follows:

```
1 if 'apple' in fruits:
2     print('Fruits has an apple')
```

Fruits has an apple

Tuple also support slicing and indexing like string and list which includes negative indexing. Some of the examples are listed below:

```
>>>fruits[::]
('apple', 'banana', 'cherry')
>>>fruits[1:3]
('banana', 'cherry')
>>>fruits[::-1]
('cherry', 'banana', 'apple')
```

Trying to update an item of tuple will raise a `TypeError`, as tuple object doesn't support item assignment.

```
>>>fruits[0]='mango'
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    fruits[0]='mango'
TypeError: 'tuple' object does not support item assignment
```

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
>>>fruits = ('banana',)
>>>fruits
('banana',)
```

If comma is omitted Python won't recognise that object as a tuple instead considered as string class object.

```
>>>fruits = ('banana')
>>>type(fruits)
<class 'str'>
```

7.2.1 Modifying a Tuple

Tuples are ordered collections of elements, similar to lists. However, unlike lists, tuples cannot be

modified after creation. This means you can't change the values of individual elements within a tuple. But there is a workaround and you can convert the tuple into a list, change the list, and convert the list back into a tuple.

```
1 x = ("apple", "banana", "cherry")
2 y = list(x)
3 y[1] = "kiwi"
4 x = tuple(y)
5 print(x)
```

```
('apple', 'kiwi', 'cherry')
```

7.2.2 Adding Items

Since tuples are immutable, directly you can't add an item to it. But there are other ways to add items to a tuple, by converting a tuple to list we can add item to it and later convert it to a tuple, as follows:

```
1 fruits = ("apple", "banana", "cherry")
2 l1 = list(fruits)
3 l1.append("orange")
4 fruits = tuple(l1)
5 print(fruits)
```

```
('apple', 'banana', 'cherry', 'orange')
```

Another way we can do by adding a tuple to another tuple instead of converting to a list.

```
>>>t1 = (2,3)
>>>t2 = (4,)
>>>t1 = t1+t2
>>>print(t1)
(2, 3, 4)
```

7.2.3 Methods

Tuple objects supports only two built-in methods:

- **count()** - Return the number of times that value appears in the tuple
- **index()** - Search for the first occurrence of the value, and return its position. Returns `ValueError`, if the given value is not present.

```
>>> t = (1,2,3,4,4,5)
>>> t.count(4)
2
>>> t.index(4)
3
>>> t.index(110)
```

Traceback (most recent call last):

```
File "<stdin>", line 1, in <module>
ValueError: tuple.index(x): x not in tuple
```

7.2.4 Packing and Unpacking

Packing and unpacking tuples in Python are convenient ways to work with multiple values. Here's a breakdown of both concepts:

Packing refers to creating a tuple by assigning multiple values within parentheses `()` and the order you specify the values becomes the order they are stored within the tuple.

```
>>>a, b, c = 1, 2, 3
>>>t = (a,b,c) #Packing to an object
>>>type(t)
<class 'tuple'>
```

In this example `t` become a tuple containing three integers 1,2,3. While Unpacking assigns individual values from a tuple to separate variables. The number of variables must match the number of elements in the tuple.

```
>>>a1, b1, c1 = t #unpacking an object
>>>print(a1, b1, c1)
1 2 3
```

You can unpack tuples with more or less elements than variables using extended unpacking with an asterisk `*`. Unpacking with `*` before a variable name captures all remaining elements in a list.

```
>>> fruits = ("apple", "banana", "cherry", "mango", "orange")
>>> first_fruit, second_fruit, *other_fruits = fruits
>>> print(first_fruit)
apple
>>> print(second_fruit)
banana
>>> print(other_fruits)
['cherry', 'mango', 'orange']
```

7.2.5 Why tuple?

Tuples are a data structure in Python, and they have various use cases where they are preferable over other data structures like lists or dictionaries. Tuples are similar to lists in that they can store multiple items, but they have some key differences:

- **Immutable Data:** Tuples are immutable, meaning their elements cannot be modified after creation. This immutability makes tuples suitable for use cases where data should not change.
- **Hashable:** Tuples are hashable, which means they can be used as keys in dictionaries, while lists cannot.

Here are some common use cases when tuples are preferable:

- **Return Multiple Values from Functions:** Tuples are often used to return multiple values from a function when you want to ensure that the values are treated as a single unit, discussed in Chapter 8.
- **Unpacking:** Tuples can be used for unpacking values returned from functions or when iterating through sequences, just discussed in Section 7.2.4.
- **Dictionary Keys:** Since tuples are hashable, they can be used as keys in dictionaries when the key is a combination of multiple values, discussed more in details later in this Chapter.

```
>>> coordinates = {(1, 2, 3): "Point A", (4, 5, 6): "Point B"}
>>> coordinates[(1,2,3)]
'Point A'
```

- **Records or Structs:** Tuples can represent a fixed collection of related data fields, similar to a lightweight struct or record.
- **Immutability:** When you need to ensure that data remains constant throughout the program, using tuples can prevent accidental modifications.
- **Performance:** Tuples can be slightly faster than lists when it comes to iteration and indexing since they are immutable and have a fixed structure.
- **Data Integrity:** Tuples can be used to enforce data integrity by preventing the accidental modification of values within a collection.
- **Function Arguments:** Tuples can be used to pass multiple arguments to functions without explicitly defining named parameters. Discussed in Chapter 8.

Remember that the choice between tuples, lists, and other data structures depends on the specific requirements of your program and the nature of the data you are working with. Tuples are particularly useful when you need a fixed, unchangeable collection of items.

7.2.6 Tuple Comprehension

In Python, there is no direct tuple comprehension construct like there is for list comprehensions. However, you can create a tuple using a generator expression and the `tuple()` constructor to convert it into a tuple. A generator expression is similar to a list comprehension, but it returns an iterable generator object instead of a list. Here's the basic structure of a tuple comprehension using a generator expression:

```
tuple_variable = tuple(expression for element in iterable)
```

The meaning of each component in the above syntax is summarised below:

- **expression:** The expression to evaluate for each element in the iterable.
- **element:** The variable that takes on each value from the iterable.
- **iterable:** The iterable you want to loop through to create the tuple.

Here's an example of creating a tuple using a generator expression:

```
>>> numbers = [1, 2, 3, 4, 5]
>>> squared_tuple = tuple(x ** 2 for x in numbers)
>>> print(squared_tuple)
(1, 4, 9, 16, 25)
```

7.3 Set

In Python, a set is a built-in data structure used to store a collection of unique and unordered elements. Items of the set are immutable type objects and sets are defined by enclosing a comma-separated sequence of elements within curly braces `{}` or by using the built-in `set()` constructor method. Once a set is created set items are not modifiable, but you can add new items using `.add(elem)` method. Let us first create a list by using comma-separated sequence of elements:

```
>>>my_set = {1, 2, 3, 4, 5}
>>>my_set
{1, 2, 3, 4, 5}
```

Note that you should not use empty curly braces to create an empty set because Python will interpret it as a dictionary.

```
>>>s = {}
>>>type(s)
<class 'dict'>
```

The `set()` constructor allows you to create an empty set. Items can be added later using the `add()` method. As shown in the next illustration

```
>>>s = set()
>>>type(s)
<class 'set'>
>>>s.add(1)
>>>s.add(2)
>>>s
{1, 2}
```

Remember adding an item is not ordered in a set like a list. It can be placed anywhere of a given object and like list, set doesn't keep duplicate items. Python simply ignored the item and keep the first item. Now a duplicate item to set `s`, that we have created in the previous illustration.

```
>>>s.add(2) #adding a duplicate item
>>>s
{1, 2}
```

While creating a set using `set()` constructor method, you can pass an iterable (e.g., a list or tuple) object as an argument. This will create a set with the unique elements from the iterable. For example:

```
>>>my_list = [1, 2, 2, 3, 4, 4, 5]
>>>my_set = set(my_list)#duplicate items rejected
>>>my_set
{1, 2, 3, 4, 5}
```

Similar to the list comprehensions, you can use set comprehensions to create sets by applying an expression to each element of an iterable. For example:

```
>>>my_list = [1, 2, 3, 4, 5]
>>>my_set = {x * 2 for x in my_list}
>>>my_set
{2, 4, 6, 8, 10}
```

Like list and tuple using **in** and **not in** operators can be used to test the membership of any item and **len()** function returns the number of elements in a set:

```
>>>len(my_set)
5
>>>2 in my_set
True
>>>11 in my_set
False
```

Sets items are not accessible using index like list or tuple objects. Sets items are iterate through loop using for loop as follows:

```
>>> s = {1,2,3}
>>> for item in s:
...     print(item, end=' ')
...
1 2 3
```

7.3.1 Operations on Set

Union of two sets: Set operations in Python can be performed in two different ways: by operator or by method. Let's take a look at how these operators and methods work, using set union as an example. Given two sets, **set1** and **set2**, the union of **set1** and **set2** is a set consisting of all elements in either set. Consider these two sets:

```
>>>set1 = {1,3,4,5,6}
>>>set2 = {1,7,3,10}
```

In Python, set union can be performed with the union (**|**) operator:

```
>>>set1 | set2
{1, 3, 4, 5, 6, 7, 10}
```

Note that elements 1 and 3, which appears in both the sets, appears only once in the union operation. Set union can also be obtained with the **union()** method. The method is invoked on one of the sets, and the other is passed as an argument:

```
>>>set1.union(set2)
```



```
{1, 3, 4, 5, 6, 7, 10}
```

The working of union operator (`|`) and `union()` method is quite different. For union operator `|`, both the operands must be of set type, but `union()` method can accept any iterable type as argument, later converts that iterable as set and perform union operation. See the next illustration:

```
>>>num1 = {1,3,4,5,6}
>>>num2 = tuple([11,2,3])
>>>num1|num2
Traceback (most recent call last):
  File "<pyshell#31>", line 1, in <module>
    num1|num2
TypeError: unsupported operand type(s) for |: 'set' and 'tuple'
>>>num1.union(num2)
{1, 2, 3, 4, 5, 6, 11}
```

Python raised `TypeError` if any of the operand is a different type of object, other than set. But `union()` can take any type of iterable object as argument and perform the union operation. More than two sets may be specified with either the operator or the method:

```
>>>a = {1, 2, 3, 4}
>>>b = {2, 3, 4, 5}
>>>c = {3, 4, 5, 6}
>>>a.union(b, c)
{1, 2, 3, 4, 5, 6}
>>>a | b | c
{1, 2, 3, 4, 5, 6}
```

Intersection between two sets: `a.intersection(b)` and `a & b` return the set of elements common in both sets `a` and `b`. You can also specify multiple sets with the intersection method or operator, just like you can with set union operation. If there are no common elements between the sets, the intersection will be an empty set `set()`.

```
>>>a.intersection(b)
{2, 3, 4}
>>>a & b
{2, 3, 4}
>>> c = {6,7}
>>> a & c
>>> a & c #returns an empty set
set()
```

Like set union method and operator, the `intersection()` method and intersection (`&`) operator can handle intersections of more than two sets at once.

Difference between two sets: Set difference finds elements that are in the first set but not in the second. `a.difference(b)` and `a-b` is the required syntax to get the set of all elements that are in `a` but not in `b`:

```
>>>a.difference(b)
```

```
{1}
>>>a-b
{1}
```

Another way to think of this is that `a.difference(b)` and `a-b` return the set, that results when elements are present in `a` not in `b`. In simpler terms it removes the “overlap” between `a` and `b`. Once again, you can specify more than two sets, when multiple sets are specified, the operation is performed from left to right. In the example below, `a - b` is computed first, resulting in `{1, 2, 3, 300}`. Then `c` is subtracted from that set, leaving `{1, 2, 3}`.

```
>>>a = {1, 2, 3, 30, 300}
>>>b = {10, 20, 30, 40}
>>>c = {100, 200, 300, 400}
>>>a-b-c
{1, 2, 3}
```

Symmetric Difference: In Python, `a.symmetric_difference(b)` and `a^b` are two ways to find the symmetric difference between two sets `a` and `b`. The symmetric difference of two sets contains all elements that are in either of the sets but not in their intersection. In other words, it includes elements that are unique to each set.

```
>>>a = {1, 2, 3, 30, 300}
>>>b = {10, 20, 30, 40}
>>>a^b
{1, 2, 3, 40, 10, 300, 20}
```

The symmetric difference can also be performed using difference operator between the union and intersection operations of two sets as follows:

```
>>>(a | b) - (a & b)
{1, 2, 3, 40, 10, 300, 20}
```

Like other set operators’ symmetric difference (^) operator also allowed with multiple sets to perform symmetric difference. But `symmetric_difference()` method exactly allows only one argument.

Disjoint of two sets: Using `a.isdisjoint(b)` method we can determine `a` and `b` are disjoint set or not. Two sets are disjoint, if they have no elements in common, and if `a.isdisjoint(b)` is True, then `a & b` returns an empty set

```
>>>a = {1, 2, 3, 30, 300}
>>>b = {10, 20, 30, 40}
>>>a.isdisjoint(b)
False
>>>x1 = {1, 3, 5}
>>>x2 = {2, 4, 6}
>>>x1.isdisjoint(x2)
True
>>>x1 & x2
set()
```

Subset of a set: In set theory, a set **a** is considered a subset of another set **b**, if every element of **a** is in **b**. The required logic for subset operation is shown below:

```
a = {1, 2, 3}
b = {1, 2, 3, 4, 5}
flag = True
#For subset length of set a should be less than or equal to b
if len(a)>len(b):
    print('a is not a subset of b')

else:
    #iterating over each item of a
    for item in a:
        #checking membership in set b
        if item not in b:
            #update the flag and break from loop
            flag = False
            break
    if flag:
        print('a is subset of b')
    else:
        print('a is not a subset of b')
```

Python offers a method `issubset()` and a subset operator (`<=`) to check a set is subset of another or not.

```
>>>a.issubset(b)
True
>>>a<=b
True
```

Proper subset: The operation `a<b`, determines whether one set is a proper subset of the other. A proper subset is the same as subset, except that the sets can't be identical. A set **a** is considered a proper subset of **b**, if every element of **a** is in **b**, and **a** and **b** are not equal.

```
>>>a = {1,2}
>>>b = {1,2,3}
>>>a<b
```

```
True
```

The `<` operator is the only way to test whether a set is a proper subset or not. There is no corresponding method.

Superset: Superset operator (`>=`) is used to check a set is superset of another or not and the corresponding method is `issuperset()`. A set **b** is considered a superset of another set **a**, if **b** contains all the element of **a**.

```
>>>b>=a
True
>>>b.issuperset(a)
```

True

`a > b`, determines whether one set is a **proper superset** of the other. A set `a` is considered a proper superset of another set `b`, if `b` contains every element of `a`, and `a` and `b` are not equal.

```
>>>b>a
```

True

```
>>>a.add(3) #both sets are equal
```

```
>>>b>a
```

False

The proper superset (`>`) operator is the only way to test whether a set is a proper superset or not, there is no corresponding method.

7.3.2 Modifying a set

`a.remove(<elem>)` removes an `<elem>` from set `a`, if `<elem>` is not present, Python will raise an `KeyError` exception:

```
>>>a = {1,2,3,4,5}
```

```
>>>a.remove(1)
```

```
>>>a
```

```
{2, 3, 4, 5}
```

If you don't want to handle any exception while removing an element from a set, then use `x.discard(<elem>)`, instead of `a.remove(<elem>)`. This method quietly does nothing if `<elem>` is not in set `a`.

```
>>>a.discard(10)
```

```
>>>a
```

```
{2, 3, 4, 5}
```

```
>>>a.discard(2)
```

```
>>>a
```

```
{3, 4, 5}
```

`a.pop()` removes and returns an arbitrarily chosen element from set `a`. If set `a` is empty, `a.pop()` raises an exception:

```
>>> a.pop(), a.pop(), a.pop()
```

```
(3, 4, 5)
```

```
>>> a.pop()
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
KeyError: 'pop from an empty set'
```

Instead of removing items one by one `clear()` method removes all elements from a set. A set can be updated by adding items from other set using `update()` method or `|=` operator.

```
>>> b
```

```
{2, 3, 4, 5}
```

```
>>> b.update({6,7})
```

```
>>> b
{2, 3, 4, 5, 6, 7}
```

An in-place update of a set can be done using `intersection_update(other_set)` method or `&=` operator, that modifies a set by keeping only the elements common to both sets.

```
>>> a = {1,2,3}
>>> b = {2, 3, 4, 5, 6, 7}
>>> id(a)
2309581386496
>>> a.intersection_update(b)
>>> a
{2, 3}
>>> id(a)
2309581386496
```

An in-place modification changes the values or elements within an existing data structure (like a list, set, or object). The original data structure is mutated, and it reflects the changes made. In the above illustration `id()` on `a`, before and after modification returns same memory reference. Like `intersection_update(other_set)`, `difference_update(other_set)` method or `-=` operator modifies a set by removing elements found in the other set as in-place modification.

```
>>> a = {1,2,3}
>>> b
{2, 3, 4, 5, 6, 7}
>>> a.difference_update(b)
>>> a
{1}
```

The last method of this type is `symmetric_difference_update(other_set)` method or `^=` operator, modifies the set by keeping elements that are in exactly one of the sets.

```
>>> a = {1,2,3}
>>> a.symmetric_difference_update(b)
>>> a
{1, 4, 5, 6, 7}
```

Note, the non-operator versions of the `update()`, `intersection_update()`, `difference_update()`, and `symmetric_difference_update()` methods will accept any iterable as an argument.

7.4 Frozen Sets

Python provides another built-in type called a frozenset, which is in all respects exactly like a set, except that a frozenset is immutable. You can perform non-modifying operations on a frozenset. The inbuilt function called `frozenset()` that takes an iterable object as input and makes it immutable. Simply put, the Python Frozenset renders iterable objects unchangeable by freezing them.

```
>>> vowels = {'a', 'e', 'i', 'o', 'u'}
>>> fSet = frozenset(vowels)
>>> fSet
frozenset({'a', 'o', 'u', 'e', 'i'})
```

frozenset() takes only one parameter, that contains the elements that will be used to initialize the frozenset and returns a frozenset with items from the supplied iterable that is untouched. It returns an empty frozenset if arguments are not given.

7.4.1 Operations with frozenset

You can perform set operations such as union, intersection, and difference with other frozensets, sets, or even other iterable types (converted to sets or frozensets).

```
>>> frozen_set1 = frozenset([1, 2, 3])
>>> frozen_set2 = frozenset([2, 3, 4])

>>> frozen_set1 & frozen_set2 #intersection
frozenset({2, 3})
>>> frozen_set1 | frozen_set2 #union
frozenset({1, 2, 3, 4})
>>> frozen_set1 - frozen_set2 #difference
frozenset({1})
```

- Membership is allowed in frozenset and can be checked using **in** operator.
- You can check if one frozenset is a subset or superset of another using **issubset()** and **issuperset()** methods.
- You can get the number of elements in the frozenset using the **len()** function.
- You can convert a frozenset to a regular set, list, or tuple using the appropriate constructor.
- You can create a copy of a frozenset using the constructor or the **copy()** method.
- Since frozensets are hashable, you can use them as keys in dictionaries.
- You can iterate through the elements of a frozenset using a for loop.
- Unlike mutable sets, frozensets do not support methods like add, remove, or discard because they are immutable.
- It only requires the keys of the dictionary to produce a frozen set when you utilise a dictionary as an iterable.

```
>>> person = {'name': 'John', 'age': 23, 'sex': 'male'}
>>> frozenset(person)
frozenset({'age', 'name', 'sex'})
```

7.5 Dictionary

Python provides another composite data type called a dictionary, which is similar to a list in that it is a collection of objects. Dictionaries and lists share the following characteristics:

- Both are mutable.
- Both are dynamic. They can grow and shrink as needed.
- Both can be nested means a list and dictionary can contain another list or dictionary. A dictionary can also contain a list, and vice versa.

Dictionaries differ from lists primarily in how elements are accessed:

- List elements are accessed by their position in the list, via indexing.
- Dictionary elements are accessed via keys.

Dictionaries are Python's implementation of a data structure that is more generally known as an associative array. A dictionary consists of a collection of key-value pairs. Each key-value pair maps the key to its associated value. You can define a dictionary by enclosing a comma-separated list of key-value pairs in curly braces (`{}`). A colon (`:`) separates each key from its associated value:

```
records = {'name': 'Goutam',
           'age': 37,
           'city': 'Vellore',
           'Country': 'India'
}
```

You can also construct a dictionary with the built-in `dict()` function. The argument to `dict()` should be a sequence of key-value pairs. A list of tuples works well for this:

```
>>> records = dict([('name', 'Goutam'), ('age', 37), ('city', 'Vellore'),
...                 ('Country', 'India')])
>>> records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India'}
```

If the key values are simple strings, they can be specified as keyword arguments. So here is yet another way to define records:

```
>>> records = dict(name = 'Goutam', age = 37, city = 'Vellore', Country =
'India')
>>> records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India'}
```

The entries in the dictionary display in the order they were defined. But that is irrelevant when it comes to retrieving them. Dictionary elements are not accessed by numerical index like list. It will raise a `KeyError`, as shown below:

```
>>> records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India'}
>>> records[0]
Traceback (most recent call last):
  File "<pyshell#28>", line 1, in <module>
    records[0]
KeyError: 0
```

7.5.1 Building a Dictionary Incrementally

Defining a dictionary using curly braces and a list of key-value pairs, as shown above, is fine if you know all the keys and values in advance. But what if you want to build a dictionary on the fly? That you can start by creating an empty dictionary, which is specified by empty curly braces (`{}`). Then you can add new keys and values one at a time.

```
>>>person = {}
>>>type(person)
<class 'dict'>
```

Once an empty dictionary is created then many ways key-value pairs can be added to the dictionary. One of the methods is by using square brackets and the assignment operator (`=`). If the key does not exist, it will be added; if it does exist, the value associated with that key will be updated.

```
>>>person['fname'] = 'Joe'
>>>person['lname'] = 'Fonebone'
>>>person['age'] = 51
>>>person['pets'] = {'dog': 'Fido', 'cat': 'Sox'}
>>>person
{'fname': 'Joe', 'lname': 'Fonebone', 'age': 51, 'pets': {'dog': 'Fido',
'cat': 'Sox'}}
```

You can use the `update()` method to add multiple key-value pairs from another dictionary or an iterable of key-value pairs.

```
>>>my_dict = {}
>>>my_dict.update({'name': 'John', 'age': 30})
>>>my_dict
{'name': 'John', 'age': 30}
```

You can build a dictionary incrementally by iterating through an iterable and adding key-value pairs based on the elements of the iterable. See the illustration given below:

```
1 my_dict = {}
2 data = [('name', 'John'), ('age', 30)]
3 for key, value in data:
4     my_dict[key] = value
5
6 print(my_dict)

{'name': 'John', 'age': 30}
```

You can create a dictionary incrementally using a dictionary comprehension, which allows you to transform one or more iterables into a dictionary.


```
>>>keys = ['name', 'age']
>>>values = ['John', 30]
>>>my_dict = {k: v for k, v in zip(keys, values)}
>>>my_dict
{'name': 'John', 'age': 30}
```

You can define a function that adds key-value pairs to a dictionary and call that function to build the dictionary incrementally. More about `def` is discussed in Chapter 8.

```
1 def add_to_dict(d, key, value):
2     d[key] = value
3
4 my_dict = {}
5 add_to_dict(my_dict, 'name', 'John')
6 add_to_dict(my_dict, 'age', 30)
7 print(my_dict)

{'name': 'John', 'age': 30}
```

7.5.2 Accessing Dictionary Elements

A value is retrieved from a dictionary by specifying its corresponding key inside a pair of square brackets (`[]`):

```
>>>records['name']
'Goutam'
```

Adding an entry to an existing dictionary is simply a matter of assigning a new key and value using assignment operator:

```
>>>records['height'] = 5.56
>>>records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India',
'height': 5.56}
```

If you want to update an entry, you can just assign a new value to an existing key:

```
>>>records['city'] = 'Agartala'
>>>records
{'name': 'Goutam', 'age': 37, 'city': 'Agartala', 'Country': 'India',
'height': 5.56}
```

Retrieving the values in the sublist or subdictionary requires an additional index or key. Consider the **person** dictionary, which has children and pets as keys and values are of type list and dictionary.

```
>>>person = {'fname': 'Joe', 'lname': 'Fonebone', 'age': 51, 'spouse':
```

```
'Edna','children': ['Ralph', 'Betty', 'Joey'],
      'pets': {'dog': 'Fido', 'cat': 'Sox'}}
>>>person['children'][1]
'Betty'
>>>person['pets']['dog']
'Fido'
```

This example exhibits another feature of dictionaries: the values contained in the dictionary don't need to be the same type. In **person**, some of the values are strings, one is an integer, one is a list, and one is another dictionary. Just as the values in a dictionary don't need to be of the same type, the keys don't either. See the next example, illustrates a dictionary with different types of keys:

```
>>>foo = {42: 'aaa', 2.78: 'bbb', True: 'ccc'}
>>>foo
{42: 'aaa', 2.78: 'bbb', True: 'ccc'}
```

To delete an entry, use the **del** statement, specifying the key to delete:

```
>>>del records['height']
>>>records
{'name': 'Goutam', 'age': 37, 'city': 'Agartala', 'Country': 'India'}
```

7.5.3 Types of key's

In Python, dictionary keys can be of any hashable data type. A hashable data type is one that is both immutable and has a hash value that never changes during its lifetime. This is because dictionaries use a hash table to store their key-value pairs, and keys must be hashable to ensure efficient lookup and retrieval. Here are some common data types that can be used as dictionary keys:

- **Numeric types:** numeric data types such as integers, floating-point numbers, and complex numbers are generally considered hashable because they are immutable. As long as the value of a numeric data type remains constant, its hash value will also remain the same. Here are a few examples of numeric data types as dictionary keys:

```
# Integers as keys
int_dict = {1: "one", 2: "two", 3: "three"}
# Floats as keys
float_dict = {3.14: "pi", 2.71828: "e"}
# Complex numbers as keys
complex_dict = {1 + 2j: "complex_number_1", 3 - 4j: "complex_number_2"}
```

- We have already seen some examples of frozenset and string as dictionary key.
- You can even use built-in objects like **types** and **functions** as key:

```
>>>d = {int: 1, float: 2, bool: 3}
>>>d
{<class 'int'>: 1, <class 'float'>: 2, <class 'bool'>: 3}
```

```
>>>d[int]
1
```

- Custom Objects: You can use custom objects as dictionary keys if you implement the `__hash__()` and `__eq__()` methods to define their hash value and equality.
- It's important to note that mutable objects, such as lists and dictionaries, cannot be used as dictionary keys because they are not hashable, and their content can change, leading to unpredictable behavior. Using an unhashable object as a key will result in a `TypeError`.

```
d = {[1,2]:12}
```

```
Traceback (most recent call last):
```

```
File "<pyshell#36>", line 1, in <module>
    d = {[1,2]:12}
```

```
TypeError: unhashable type: 'list'
```

Key points related to dictionary key

- A given key can appear in a dictionary only once. Duplicate keys are not allowed. A dictionary maps each key to a corresponding value, so it doesn't make sense to map a particular key more than once.
- When you assign a value to an already existing dictionary key, it does not add the key a second time, but replaces the existing value

```
>>>MLB_team = {'Colorado' : 'Rockies', 'Boston' : 'Red Sox',
               'Minnesota': 'Twins', 'Milwaukee': 'Brewers', 'Seattle': 'Mariners'
               }
>>>MLB_team['Minnesota'] = 'Timberwolves'
>>>MLB_team
{'Colorado': 'Rockies', 'Boston': 'Red Sox', 'Minnesota': 'Timberwolves',
 'Milwaukee': 'Brewers', 'Seattle': 'Mariners'}
```

Instead of adding `'Minnesota': 'Timberwolves'` to the dictionary, it updates the associated value. Similarly, if you specify a key second time during the initial creation of a dictionary, the second occurrence will override the first:

```
>>>MLB_team = {'Colorado' : 'Rockies', 'Boston' : 'Red Sox',
               'Minnesota': 'Timberwolves', 'Milwaukee': 'Brewers',
               'Seattle' : 'Mariners', 'Minnesota': 'Twins'
               }
>>>MLB_team
{'Colorado': 'Rockies', 'Boston': 'Red Sox', 'Minnesota': 'Twins',
 'Milwaukee': 'Brewers', 'Seattle': 'Mariners'}
```

A dictionary key must be of a type that is immutable. You have already seen examples where several of the immutable types you are familiar with—integer, float, string, and Boolean—have served as dictionary keys. A tuple can also be a dictionary key, because tuples are immutable. However, neither a list nor

another dictionary can serve as a dictionary key, because lists and dictionaries are mutable:

```
>>>d = {[1, 1]: 'a', [1, 2]: 'b'}
Traceback (most recent call last):
  File "<pyshell#29>", line 1, in <module>
    d = {[1, 1]: 'a', [1, 2]: 'b'}
TypeError: unhashable type: 'list'
```

What does the error message say unhashable?

Technically, it is not quite correct to say an object must be immutable to be used as a dictionary key. More precisely, an object must be hashable, which means it can be passed to a hash function. A hash function takes data of arbitrary size and maps it to a relatively simpler fixed-size value called a hash value (or simply hash), which is used for table lookup and comparison. Python's built-in `hash()` function returns the hash value for an object which is hashable, and raises an exception for an object which isn't:

```
>>>hash('foo')
6821459173919347162
>>>hash([1, 2, 3])
Traceback (most recent call last):
  File "<pyshell#31>", line 1, in <module>
    hash([1, 2, 3])
TypeError: unhashable type: 'list'
```

All of the built-in immutable types you have learned about so far are hashable, and the mutable container types (lists and dictionaries) are not. So, for key purposes, you can think of hashable and immutable as more or less synonymous.

7.5.4 Built-in Methods

Dictionary come with a variety of built-in methods to manipulate and work with the data stored in them. Here are some common dictionary methods:

- **dict.keys():** Returns a view of all the keys in the dictionary as a type of `dict_keys`. An object of type `dict_keys` can be passed inside a `list()` to convert it a list of keys. See the illustration shown below:

```
>>>d = {'a': 10, 'b': 20, 'c': 30}
>>>d.keys()
dict_keys(['a', 'b', 'c'])
>>>list(d.keys())
['a', 'b', 'c']
```

This method can be also used in a for loop to iterate over a dictionary and get the associated value with respective key:

```
>>> d = {'a': 10, 'b': 20, 'c': 30}
>>> for key in d.keys():
...     print(d[key],end=' ')
...
10 20 30
```

- **dict.values():** Returns a view of all the values in the dictionary as a type of dict_values and can be passed inside built-in **list()** method to get all the values as a list.

```
>>> d.values()
dict_values([10, 20, 30])
```

- **dict.items():** Returns a view of all key-value pairs (tuples) in the dictionary. Instead of using keys() and values(), if we need both then items() can be used in a for loop to iterate key and value pair.

```
>>> d = {'a': 10, 'b': 20, 'c': 30}
>>> for key, value in d.items():
...     print(f'{key}:{value}',end=' ')
...
a:10 b:20 c:30
```

- **dict.get(key, default):** Returns the value for the specified key, or a default value if the key is not found.

```
>>>my_dict = {'name': 'John', 'age': 30}
>>>my_dict.get('name', 'Unknown') #returns John
'John'
>>>my_dict.get('country', 'USA') #returns default value
'USA'
```

- **dict.popitem():** Removes and returns an arbitrary key-value pair as a tuple.

```
>>>d = {'a': 10, 'b': 20, 'c': 30}
>>>d.popitem()
('c', 30)
>>>type(d.popitem())
<class 'tuple'>
>>>d
{'a': 10, 'b': 20}
```

The arbitrariness of the removed pair means that you cannot predict which key-value pair **popitem()** will return. If you need to remove the last item from a dictionary, you can use the **pop()** method by specifying the key you want to remove. Here's an example:

```
>>>d.pop('c')
```

30

```
>>>d
{'a': 10, 'b': 20}
```

- **dict.clear()**: Removes all key-value pairs from the dictionary, leaving it empty.

```
>>>d = {'a': 10, 'b': 20, 'c': 30}
>>>d.clear()
>>>d
{}
```

- **dict.copy()**: Returns a shallow copy of the dictionary. A shallow copy of a dictionary is a new dictionary that contains references to the same objects (key-value pairs) as the original dictionary, rather than creating copies of the objects themselves. Consider the dictionary given below, which contains nested dictionary as value of **original_dict**:

```
original_dict = {
    'name': 'John',
    'info': {
        'age': 30,
        'address': {
            'city': 'New York',
            'state': 'NY'
        }
    }
}
```

In the above declaration, a dictionary **info** is nested inside **original_dict** and **address** is nested inside **info**. Creating a shallow copy of it, means that changes made to the objects within the dictionary will be reflected in both the original and the shallow copy, as they still reference the same objects.

```
# Create a shallow copy of the original_dict
>>> shallow_copy = original_dict.copy()
>>> shallow_copy['name'] = 'Alice'
>>> shallow_copy['info']['age'] = 25
>>> shallow_copy['info']['address']['city'] = 'Los Angeles'
>>> shallow_copy
{'name': 'Alice', 'info': {'age': 25, 'address': {'city': 'Los Angeles',
'state': 'NY'}}}
>>> original_dict
{'name': 'John', 'info': {'age': 25, 'address': {'city': 'Los Angeles',
'state': 'NY'}}}
```

Since we have modified the nested values associated with age and city through **shallow_copy**, the values get inside the **original_dict** gets updated. Because both of the dictionary objects refers the same dictionary object and that can be verified by checking the identity of these objects. But the associated value with key **'name'** remains same in the **original_dict**.

If you don't want to modify the **original_dict**, while modifying the copied dictionary, then you need to produce a deeper copy of the **original_dict** by using **deepcopy()** method of **copy** module. An example shown below:

```
>>> import copy
>>> deep_copy = copy.deepcopy(original_dict)
>>> # Modify the deep_copy
>>> deep_copy['name'] = 'Alice'
>>> deep_copy['info']['age'] = 25
>>> deep_copy['info']['address']['city'] = 'Los Angeles'
>>> # Check the changes in the deep_copy and original_dict
>>> original_dict
{'name': 'John', 'info': {'age': 30, 'address': {'city': 'New York', 'state': 'NY'}}}
>>> deep_copy
{'name': 'Alice', 'info': {'age': 25, 'address': {'city': 'Los Angeles', 'state': 'NY'}}}
```

Before illustration of the above code, we have redefined the **original_dict** with previous key-value pairs. The recursive part of deep copying involves copying not only the top-level objects but also delving into nested objects and copying them as well. This process continues until all nested objects are duplicated, creating a complete independent copy of the entire structure. In contrast, a shallow copy of the same compound object would only copy references to the nested objects, meaning changes in the nested objects would still be reflected in both the original and the shallow copy.

- **dict.fromkeys(keys, value):** This method is used to create a new dictionary from keys and an optional value passed as parameter. The keys must be an iterable object such as list, tuple or string and if you don't provide a value in the second argument, all the keys will be assigned **None** by default.

```
>>> keys = ['name', 'age', 'country']
>>> new_dict = dict.fromkeys(keys)
>>> new_dict
{'name': None, 'age': None, 'country': None}
```

- **dict.setdefault():** method is used with dictionaries to add a new key-value pair to an existing dictionary object, if the key does not exist. Else it will return the associated value with if key exist. This method is particularly useful when you want to ensure that a key is present in a dictionary and provide a default value if it's not.

```
>>> my_dict = {'name': 'John', 'age': 30}
>>> my_dict.setdefault('country', 'USA')
'USA'
>>> my_dict
{'name': 'John', 'age': 30, 'country': 'USA'}
>>> my_dict.setdefault('country', 'India')
'USA'
```

In the above illustration, the key-value pair (**country : USA**) is added to **my_dict**, and on trying to adding a same key, the method returns the existing associated value with it. If you didn't pass the associated value, a **None** value will be added with the key. In the example listed below, the **'city'** key

did not exist, so `setdefault()` adds the key with a value of `None`.

```
>>>my_dict = {'name': 'John', 'age': 30}
>>>city = my_dict.setdefault('city')
>>>print(my_dict)
{'name': 'John', 'age': 30, 'city': None}
>>>print(city)
None
```

- `dict.update(other_dict)`: method in Python is used to update or merge the key-value pairs of one dictionary (`other_dict`) into another dictionary. It is a way to add, modify, or overwrite key-value pairs in the original dictionary with those from `other_dict`. Two dictionaries can be merged using `update()` method as follows:

```
>>>dict1 = {'name': 'John', 'age': 30}
>>>dict2 = {'city': 'New York', 'age': 31}
>>>dict1.update(dict2)
>>>print(dict1)
{'name': 'John', 'age': 31, 'city': 'New York'}
```

Values of one dictionary can be updated using the key:value pair from another dictionary using `update()` method.

```
>>>dict1 = {'name': 'John', 'age': 30}
>>>dict2 = {'age': 31, 'city': 'New York'}
>>>dict1.update(dict2)
>>>print(dict1)
{'name': 'John', 'age': 31, 'city': 'New York'}
```

You can also use the keyword arguments syntax to pass key-value pairs directly to the `update` method.

```
>>>dict1 = {'name': 'John', 'age': 30}
>>>dict1.update(city='New York', job='Engineer')
>>>print(dict1)
{'name': 'John', 'age': 30, 'city': 'New York', 'job': 'Engineer'}
```

Key Takeaway's

- Lists are created by enclosing a comma-separated sequence of elements within square brackets `[]`.
- Be cautious about aliasing (two variables referring to the same list), which can lead to unintended side effects.
- Use `list.copy()` or slicing `[:]` to create a shallow copy of a list.
- You can clear a list by assigning an empty list to it: `my_list = []`
- List supports common operations like adding, removing, searching and counting elements.
- Lists can be memory-intensive when they contain many elements. In such cases, consider using generators or other data structures.

- Lists are suitable for storing collections of items when you need to maintain order, and you need the ability to modify the collection during the program's execution
- Lists are indexed by integers, while dictionaries are indexed by keys.
- Lists allow duplicates and have an order, while sets do not allow duplicates and have no order.
- Lists are like tuples but are mutable, whereas tuples are immutable.
- Sets are created using curly braces `{}` or the `set()` constructor method. But a pair of empty curly braces `{}` creates a dictionary, instead of a set.
- Sets are mutable, meaning you can add and remove elements.
- Sets do not support indexing, slicing, or other sequence operations because they are unordered.
- Sets are commonly used for eliminating duplicates and performing mathematical set operations.
- A frozenset is an immutable version of a set and created using the `frozenset()` constructor method.
- Frozensets are hashable and can be used as keys in dictionaries, unlike sets.
- Frozensets are useful when you need an immutable collection of unique elements and when you want to use them as keys in dictionaries.
- A dictionary is an unordered collection of key-value pairs are created using curly braces `{}` or the `dict()` constructor method.
- Dictionary keys must be unique, and they are case-sensitive.
- Values in a dictionary do not need to be unique, and the same value can be associated with multiple keys.
- In Python versions prior to 3.7, dictionaries were unordered, meaning the order of key-value pairs was not guaranteed. In Python 3.7 and later, dictionaries maintain insertion order.
- Dictionaries are used to store and retrieve data efficiently, and they are suitable for situations where you need to associate keys with values.

Practice Questions

1. Analyse a stock portfolio by storing stock symbols, quantities, and prices in tuples, and then calculate total portfolio value, average price, maximum stock price, minimum stock price, total stock. Consider the given portfolio:

```
portfolio = [("AAPL", 10, 150.0), ("GOOGL", 5, 2700.0),
             ("TSLA", 8, 700.0),
             ("AMZN", 4, 3500.0),
            ]
```

2. Take a list of numbers as input from user and finds the sum and product of all the numbers.
3. Take a list of numbers as input from user and find the maximum and minimum values and print them without using `min()` and `max()` function.
4. Write a program to manage student records with attributes like name, age, and CGPA. Store each student's information as a tuple in a list and display the student record with highest CGPA.
5. Take a list of numbers as input and calculates mean, median, mode for a list of numbers.

Hint: The **median** of a list of numbers is the "middle" value when the list is ordered from least to greatest. It represents the point where half the numbers in the list are smaller, and the other half are larger. For odd number of items, the median is the middle number and for even number of items

median is the average of the two middle numbers. The **mean** of a list of numbers, also called the average.

The **mode** of a list of numbers is the number that appears most frequently within the list. There can be one mode, multiple modes, or even no mode at all in a given list.

6. Create a program that sums up even numbers from a given list of integers. Use continue to skip odd numbers and break if the sum reaches 100 or more.
7. Given a list of names, write a program that continues to ask the user for a name until it finds a match in the list. If a match is found, break the loop and print the position of the name in the list.
8. Write a program to take a string as input and form a dictionary, that will have every character as key and frequency count of each character will be the value. See the sample input output.

Input: "Mississippi"

Output: {'M': 1, 'i': 4, 's': 4, 'p': 2}

9. Write a program to take a string with repeated characters as input and compress the string by converting repeated characters into tuples containing the character and the number of repetitions as a list (e.g., "aaabbbbcc" becomes [('a', 3), ('b', 4), ('c', 2)]).
10. Create a dictionary containing the information about subject's code and number. Subject codes will be the key and score will be the associated value. See the dictionary as given below and perform the operations as listed from (a) – (f).

subjects = {"MATH101": 85, "PHYS202": 92, "CS303": 78}

- (a) Add the details of two other subjects as "CSE202": 67, "INT304": 56.
- (b) Check a subject code already exists or not, if already there, then display a message 'Already Added', else add the new subject.
- (c) Display the subject name that have highest marks
- (d) Calculate the average and total marks
- (e) Design the functionality to remove any subject from the dictionary
11. Write down a program and take two lists of integers from user as input and find the followings:
 - (a) Prepare a third list which have all the unique numbers after removing the duplicate items
 - (b) Prepare a third list having elements that are present in both lists.
 - (c) Prepare a list that will have the elements present in the first list but not in the second
 - (d) Prepare a list that have elements that are in either list but not in both.
 - (e) Print True, if the list has no common elements, else False.
12. Prepare a python script that will take two sets as input from the user and display a dictionary containing the following set operations:
 - (a) Union of set1 and set2.
 - (b) Intersection of set1 and set2.
 - (c) Difference of set1 minus set2.
 - (d) Symmetric difference of set1 and set2.

The dictionary should have keys "union", "intersection", "difference", and "symmetric_difference".

13. Write a Python script that takes two dictionaries as input and returns a new dictionary containing all the key-value pairs from both dictionaries. If there are overlapping keys, the values from dict2 should overwrite those from dict1.

14. Create a python script that takes a dictionary as input and returns a new dictionary where the keys and values are swapped. The input dictionary should be assumed to have unique values.

*****END OF CHAPTER*****